darkmpc, Milestone 3 walkthrough, part 2

Separate processes, TLS, and the first benchmarks

Tave La Rocque

4 November 2026

Milestone 3, part 2: separate processes, TLS, and the first benchmarks

What was built

The crate `darkmpc-server` runs each of the three servers as its own operating-system process. The servers talk over mutually authenticated TLS 1.3, using rustls with the ring provider. The protocol code is unchanged from part 1: the transport sits behind the core's `Net` interface. Four commands cover a full run. `setup` creates a deployment certificate authority and one certificate per server. `deal` stands in for the traders: it generates a random batch and writes one share file per server. `run` starts one server. `check` reconstructs the fills from the three outputs and compares them with the reference matcher. `scripts/bench_local.sh` runs all four with the three servers on one machine, and `scripts/bench_grid.sh` produces the table below. Every benchmark run is also a correctness check: the script fails if the fills differ from the reference.

The links

Each ordered pair of servers gets its own TLS connection, so server i has two outbound connections, used only for sending, and two inbound, used only for receiving. Splitting by direction avoids a classic deadlock. In multiplication all three servers send before they receive. If each wrote straight into a socket, all three could block on full buffers, each waiting for a reader that is itself blocked writing. Instead every sending half is a writer thread fed by a queue, so the protocol never waits on a send.

Authentication is layered. Every certificate is signed by the deployment CA. The connecting side additionally checks that the peer's certificate names the server it meant to reach. Both sides then pin the exact certificate bytes they expect for each peer, so a connection only counts as "server j" if it presents server j's certificate. A server that connects twice claiming the same identity is rejected.

Once connected, the servers agree on the pairwise keys for the pseudorandom streams. Server i draws k_{i+1} from the operating system's randomness, sends it to server $i + 1$ over their TLS link, and receives k_i from server $i - 1$. That reproduces exactly the key layout the core uses: each key known to two servers.

A bug that only a real network could show

The first WAN run at 100,000 orders crashed with “connection reset by peer”. The cause sits in how TCP and TLS 1.3 interact, and it is worth understanding.

After a TLS 1.3 handshake, the server side of a connection sends session tickets: small messages that would let the client resume later. On our outbound connections the connecting side only writes and never reads, so those tickets sit unread in its receive buffer. When a process closes a socket that still has unread incoming data, TCP sends a reset rather than a normal close. RFC 1122 (Section 4.2.2.13) says that if a host “issues a CLOSE call while received data is still pending in TCP … its TCP SHOULD send a RST to show that data was lost”, and Linux behaves this way. A reset is a blunt instrument: the other side’s reads fail, and data it had not yet read is lost. That is exactly what we observed.

In the WAN simulation, receivers deliberately wait before reading, so the slowest server still had protocol data buffered when a faster server finished and exited. The reset destroyed that data, and the slow server crashed mid-protocol. In-memory channels have none of this, so no earlier test could catch it.

Two changes fix it. The servers no longer send session tickets, since we never resume sessions. And every server now passes a final barrier: it sends an empty message to both peers and waits for theirs before closing. A peer sends its barrier message only after finishing the protocol, which means after reading everything sent to it. So no server closes until both peers have consumed all of its data. Three consecutive WAN runs at 100,000 orders then completed with correct fills.

Simulating a network honestly

On one machine, real loopback latency is microseconds, which says nothing about deployment. So each link can simulate a pipe with a fixed one-way latency and a bandwidth cap. The sender works out when the message would finish arriving: it starts when the link is free, takes (bytes ÷ bandwidth) to transmit, then travels for the one-way latency. It stamps that time on the message, and the receiver waits until then before handing it to the protocol. Messages queue behind each other as on a real link. A test checks that the simulation changes timing and never results. Two profiles are used. LAN is 0.47 ms round trip and 10 Gbit/s, matching the ping reported for the 2018 paper’s LAN. WAN is 40 ms round trip and 1 Gbit/s, roughly a cross-continent link.

First measurements

Median of three runs. Each batch spreads 3 elimination passes over orders on 100 distinct symbols in a 13-bit symbol field, and every run’s fills match the reference. These were measured in my development workspace: all three servers on one machine with **2 vCPUs**, so the servers compete for CPU. That understates what three separate machines would do at large batch sizes.

Profile	Orders	Rounds	MB sent per server	Auction time	Orders/s
LAN	100	292	0.6	0.092 s	1,082
LAN	1,000	316	6.9	0.117 s	8,576
LAN	10,000	348	74.6	0.380 s	26,316
LAN	100,000	372	804.1	4.645 s	21,528
WAN	100	292	0.6	5.891 s	17
WAN	1,000	316	6.9	6.447 s	155
WAN	10,000	348	74.6	7.905 s	1,265
WAN	100,000	372	804.1	17.616 s	5,676

The numbers can be explained from first principles, which is a good check that they are real.

On the WAN, every round costs at least one one-way latency, 20 ms. At 100 orders, 292 rounds × 20 ms = 5.84 s, and the measurement is 5.89 s: latency is essentially everything. At 100,000 orders the round floor is 372 × 20 ms ≈ 7.4 s. Each server pushes 804 MB through two 1 Gbit/s links, which adds a few more seconds of transmission. Then comes computation. Together that is the 17.6 s measured.

On the LAN, latency is small, so small batches are dominated by fixed per-round overhead, and large batches by computation and bandwidth. Throughput falls between 10,000 and 100,000 orders because two vCPUs are running three servers that each want a core.

The design point shows clearly. Rounds grow only with the logarithm of the batch: from 292 at 100 orders to 372 at 100,000. So on a WAN, a batch 1,000 times larger costs only about three times as long.

What these numbers are not

They are **semi-honest** numbers. They must never be set against the 2018 paper's actively secure figures as if like for like. Its three-party volume match handled 1,000 orders in 0.900 s online, plus 4.136 s of preprocessing, on a real LAN with malicious security. That is a different security model, a different mechanism, and three real machines against one shared 2-vCPU box. The like-for-like comparison waits for Milestone 4's malicious version, run on three real machines.

Next: real machines

The binary already runs on three hosts: run `setup` and `deal` once, copy the directory to each machine, and start each server with the real addresses and no simulation flags. The cloud benchmarks need three identical VMs in one region for the LAN numbers, and VMs in two or three regions for the WAN numbers.